



Discrepancy Learning for Dynamical System Identification

Dávid Gavallér
ES02RO

May 12, 2026

Abstract

This document presents an investigation into discrepancy learning methods for nonlinear dynamical systems using Sparse Identification of Nonlinear Dynamics (SINDy) and Neural Network based correction models. Several systems were examined, including a linear spring-mass-damper system, a falling ball with nonlinear aerodynamic drag, and the nonlinear Van der Pol oscillator. The systems were examined in MATLAB, where simulated measurement data was generated and used for system identification and discrepancy learning.

The objective of the work was to:

1. Reconstruct governing differential equations from simulated measurement data with SINDy to understand it
2. Investigate the effect of measurement noise and numerical differentiation
3. Apply sparse regression techniques to identify compact physical models
4. Introduce artificial discrepancies into physical systems
5. Learn and compensate these discrepancies using both SINDy and Neural Networks

The results demonstrate that SINDy can successfully recover interpretable governing equations, while discrepancy learning methods estimate and compensate for the difference between the nominal model and the observed system dynamics.

1 Introduction

Dynamic systems are traditionally modeled using first principles physics equations derived from fundamental laws. However, these models often fail to capture all the complexities of real-world systems because of numerous reasons:

- Simplifications
- Aging effects
- Unknown physical phenomenas

This leads to discrepancies between model predictions and observed behavior. Discrepancy learning methods aim to bridge this gap by learning the difference between the nominal model and the true system dynamics from data.

Data-driven system identification methods aim to reconstruct system dynamics directly from measured data. One important approach is Sparse Identification of Nonlinear Dynamics (SINDy) [1], which identifies governing equations using a predefined function library. The core assumption behind SINDy is that physical systems can be represented compactly using a set of candidate functions.

In addition to direct system identification, SINDy can be used for discrepancy learning by modeling the difference between a nominal model and observed dynamics alongside Neural Networks.

This project investigates the possibilities with SINDy for both system identification and discrepancy learning on several example systems, including a linear spring-mass-damper system, a falling ball with nonlinear drag, and the Van der Pol oscillator. Also, the results achieved with SINDy are compared to those obtained using Neural Networks for discrepancy learning.

2 Theoretical background

In this section, the theoretical background of the methods used in this project is reviewed.

2.1 Sparse Identification of Nonlinear Dynamics (SINDy) [1]

SINDy is a method for discovering the equation of a system from data. It assumes that the system can be described as a combination of a few candidate functions.

The time evolution of a dynamical system is generally expressed as:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)) \quad (1)$$

where $\mathbf{x}(t)$ is the state vector at time t and $\mathbf{f}$ is the function describing the system dynamics.

In order to determine $\mathbf{f}$, from data, measurements of $\mathbf{x}(t)$, and if possible, measurements of $\dot{\mathbf{x}}(t)$ are collected. The time derivatives can also be estimated from the state measurements using numerical differentiation techniques.

This gives the following two matrices:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^T(t_1) \\ \mathbf{x}^T(t_2) \\ \vdots \\ \mathbf{x}^T(t_m) \end{bmatrix} = \begin{bmatrix} x_1(t_1) & x_2(t_1) & \cdots & x_n(t_1) \\ x_1(t_2) & x_2(t_2) & \cdots & x_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(t_m) & x_2(t_m) & \cdots & x_n(t_m) \end{bmatrix} \quad (2)$$

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{\mathbf{x}}^T(t_1) \\ \dot{\mathbf{x}}^T(t_2) \\ \vdots \\ \dot{\mathbf{x}}^T(t_m) \end{bmatrix} = \begin{bmatrix} \dot{x}_1(t_1) & \dot{x}_2(t_1) & \cdots & \dot{x}_n(t_1) \\ \dot{x}_1(t_2) & \dot{x}_2(t_2) & \cdots & \dot{x}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{x}_1(t_m) & \dot{x}_2(t_m) & \cdots & \dot{x}_n(t_m) \end{bmatrix} \quad (3)$$

Next, a library of candidate functions $\Theta(\mathbf{X})$ is constructed, which includes various nonlinear functions of the state variables. For example, the library may include:

$$\Theta(\mathbf{X}) = [1 \quad \mathbf{x}_1 \quad \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_n \quad \mathbf{x}_1^2 \quad \cdots \quad \mathbf{x}_n^2 \quad \mathbf{x}_1\mathbf{x}_2 \quad \cdots] \quad (4)$$

As it can be seen, there is a lot of freedom in the choice of the candidate functions, and it is important to include functions that are relevant to the system being studied based on prior knowledge.

To determine the active functions in the library, a regression problem is set up to determine the vectors of coefficients Ξ such that:

$$\dot{\mathbf{X}} \approx \Theta(\mathbf{X})\Xi \quad (5)$$

The least-squares solution to this problem is given by:

$$\Xi = (\Theta^T \Theta)^{-1} \Theta^T \dot{\mathbf{X}} \quad (6)$$

To promote sparsity in the solution, a thresholding step is applied to set small coefficients to zero, then the coefficient are recalculated using only the remaining active functions. This process is repeated until there is no change in the active functions.

3 Spring-Mass-Damper System

The first system investigated was a simple linear spring-mass-damper system, since it is a well-known system with a simple governing equation, easy to test SINDy on. The system consists of a mass m attached to a spring with spring constant k and a damper with damping constant c . The system is illustrated in Figure 1.

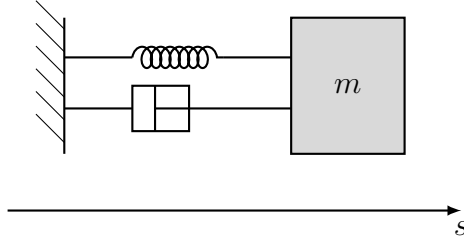


Figure 1: Mass-spring-damper system

The MATLAB implementation of the system started off from Newton's second law:

$$F = ma \quad (7)$$

Knowing that the forces acting on the mass are the spring force $F_s = -ks$ and the damping force $F_d = -cv$, the governing equation of the system can be written as:

$$-k \cdot s - c \cdot v = m \cdot a \quad (8)$$

This can be rearranged to express the acceleration:

$$a = -\frac{k}{m} \cdot s - \frac{c}{m} \cdot v \quad (9)$$

Next, by choosing specific values for the parameters m , k , and c , the ideal system was simulated in MATLAB to generate measurement data for s , as in a real scenario, it is most likely that only the position of the mass can be measured, the velocity and acceleration would have to be estimated from the position data.

The chosen parameters, initial conditions and simulation settings were:

- Mass $m = 1$ [kg]
- Spring constant $k = 4$ [N/m]
- Damping constant $c = 0.8$ [Ns/m]
- Initial position $s(1) = 1$ [m]
- Initial velocity $v(1) = 0$ [m/s]
- Simulation time $t = 0$ to 15 [seconds] with a time step of 0.01 [seconds]

The plot of the idealsimulated position, velocity and acceleration data is shown in Figure 2.

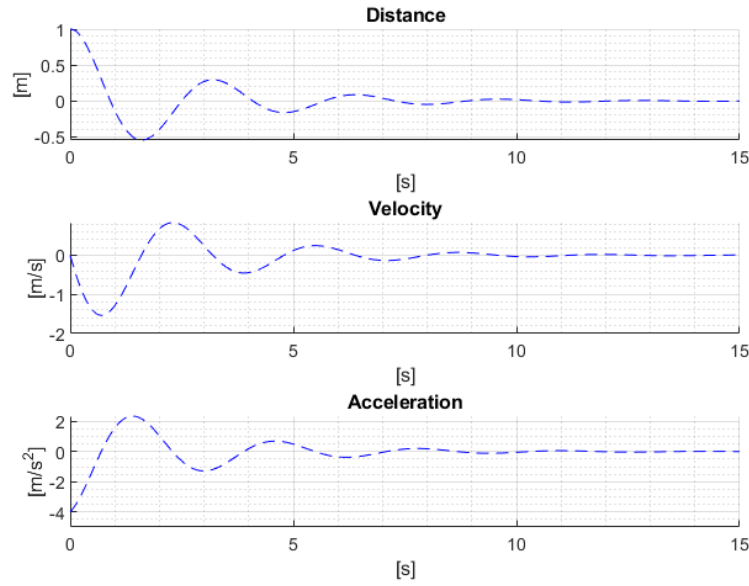


Figure 2: Simulated ideal position, velocity, and acceleration data for the spring-mass-damper system

Next, noise was added to the position data (gaussian with a noise level of 0.00001) to simulate measurement noise, the velocity and acceleration were estimated from the noisy position data using the `diff()` function, which calculates the difference between consecutive elements. The same principle was applied to the velocity data to estimate the acceleration. The results are shown in Figure 3.

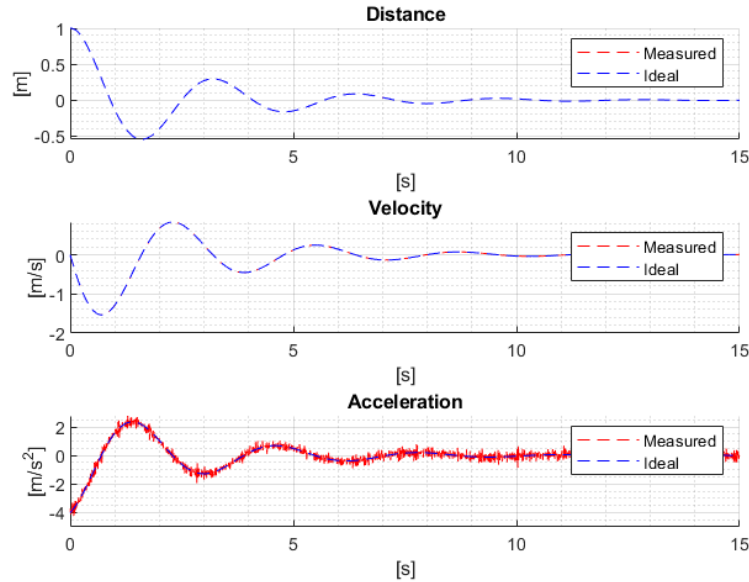


Figure 3: Simulated noisy position, velocity, and acceleration data for the spring-mass-damper system

As it can be seen, even this little noise becomes significant in the acceleration data, because of the numerical differentiation.

Next, it was time to test SINDy on the noisy "measurement" data. A library of candidate functions was constructed:

$$\Theta(\mathbf{X}) = [1 \quad s \quad v \quad vs \quad v^2 \quad s^2 \quad s^2v \quad sv^2 \quad s^3 \quad v^3] \quad (10)$$

The library was normalized with the RMS of each column for the Ξ coefficients to be comparable, and the regression was applied to determine the coefficients for each candidate function.

This way, the results for Ξ are the following (of course, the results may differ on each run because of the noise):

$$\Xi = \begin{bmatrix} 0.0001 \\ -0.3377 \\ -0.8990 \\ -0.0046 \\ -0.0007 \\ -0.0025 \\ -0.0048 \\ -0.0054 \\ -0.0046 \\ 0.0015 \end{bmatrix} \quad (11)$$

References

- [1] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. “Discovering governing equations from data by sparse identification of nonlinear dynamical systems”. In: *Proceedings of the National Academy of Sciences* 113.15 (Mar. 2016), pp. 3932–3937. ISSN: 1091-6490. DOI: 10.1073/pnas.1517384113. URL: <http://dx.doi.org/10.1073/pnas.1517384113>.